



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Generative AI for writing (research) software

Dr. Inga Ulusoy, Scientific Software Center
June 2026

Generative AI for software development

- Specifically trained models that understand programming languages, intent, and the software development process
- Trained on millions of public, open-source software repositories
- Distinguish between **provider**, **tool** and **model**

Provider	OpenAI	Anthropic	Google	GitHub
Tool	Codex	Claude	Antigravity	VSCode
Models	GPT-4.x, GPT-5.x	Sonnet, Haiku, Opus	Gemini	“all”

Providers

- These are the creators / hosts of the models and interfaces / tools
 - Anthropic
 - OpenAI
 - GitHub Copilot
 - Cursor
 - Google
 - Alibaba
- Differences in models and tools
- Differences in pricing and data management
- Differences in speed and reliability

Free
A fast way to get started with GitHub Copilot.
\$0 USD
[Get started](#)
[Open in VS Code](#)

Pro
Accelerate workflows with GitHub Copilot.
\$10 USD
per user / month
[Upgrades are paused as we roll out a flexible billing experience. We know this interrupts your flow and appreciate your patience while we get this right. \[Learn more\]\(#\)](#)

Pro+
Scale with agents and more models.
\$39 USD
per user / month
[Upgrades are paused as we roll out a flexible billing experience. We know this interrupts your flow and appreciate your patience while we get this right. \[Learn more\]\(#\)](#)

Codex Pricing

Codex is included in your ChatGPT Free, Go, Plus, Pro, Business, Edu, or Enterprise plan

[Copy Price](#)

Individual Business / Enterprise

Free
Explore Codex capabilities on quick coding tasks.
\$0 /month
[Get Free](#)

Plus
Power a few focused coding sessions each week.
\$20 /month
[Get Plus](#)

Go
Use Codex for lightweight coding tasks.
\$8 /month
[Get Go](#)

Choosing a Claude plan

Pro Choose 5x or 20x high March 16, 2026 From \$100 /month Use the following guide to determine which plan is right for you.				
Plan	Price	Billing Interval	Usage Capacity	Best For
Free	\$0	N/A	Limited	Occasional use
Pro	\$20/month \$200/year	Monthly or annual	Standard	Regular use
Max 5x	\$100	Monthly	5x Pro capacity per session	Frequent users who work with Claude on a variety of tasks
Max 20x	\$200	Monthly	20x Pro capacity per session	Daily users who collaborate often with Claude for most tasks

The plans listed above are intended for individual users. Looking for a plan for your organization or company? Read more about our Team and Enterprise plans:

- [What is the Team plan?](#)
- [What is the Enterprise plan?](#)



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Models

- The underlying “brain”
- Trained on extensive codebases
- To see the current state:
AI / LLM Leaderboards

Top Coding Models in Kilo

[All Models](#) [Compare Models](#) [Browse All 500+ Models](#) [Open Source Models](#) [PinchBench](#)



Top Coding Models in Kilo This Week

Our picks based on real-world testing • [View usage stats](#)

Claude Opus 4.6
by Anthropic

#1

Qwen 3.6 Plus
by Qwen

#2

GPT 5.4
by OpenAI

<https://kilo.ai/leaderboard>

- Actual performance
depends on task & tool



SCIENTIFIC
SOFTWARE
CENTER

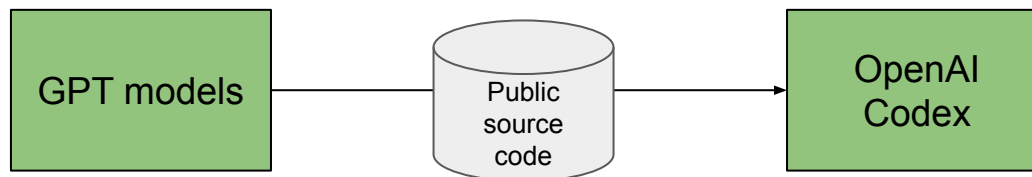


UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

MODEL	Rating conservative	Price \$/M 4.1 blended	Context window	Speed tokens charts	TTFT latency	Arena Avg 7 arenas expand	SWE-Bench Verified 89 models	LiveCodeBench 71 models	H
1 Claude Mythos Preview	57.3	—	—	—	—	—	93.9%	—	—
2 GPT-5.5	53.1	\$10.00	1.1M	24 o/s	9.3s	1646	—	—	—
3 Claude Opus 4.7	51.6	\$9.00	1M	46 o/s	1.5s	1892	87.6%	—	—
4 Claude Opus 4.6	45.6	\$9.00	1M	121 o/s	1.7s	2011	80.8%	—	—
5 Kimi K2.6	45.6	\$1.56	262.1k	25 o/s	35.1s	1254	80.2%	—	—
6 GLM-5.1	45.1	\$2.00	200k	—	—	1539	—	—	—
7 DeepSeek-V4-Pro-Max	45.0	\$2.09	1.0M	71 o/s	89.5s	1080	80.6%	93.5%	—
8 GPT-5.4	44.3	\$5.00	1M	135 o/s	988ms	1716	—	—	—
9 Gemini 3.1 Pro	44.1	\$5.00	1.0M	90 o/s	—	2093	80.6%	—	—
10 GPT-5.3 Codex	43.9	\$4.20	400k	141 o/s	1.3s	1242	—	—	—
11 Qwen3.6 Plus	43.3	\$1.00	1M	88 o/s	59.1s	1207	78.8%	—	—
12 Claude Opus 4.5	41.0	\$9.00	200k	287 o/s	1.4s	1617	80.9%	—	—
13 GPT-5.2 Codex	40.2	\$4.20	400k	163 o/s	7.4s	1160	—	—	—
14 MiniMax M2.7	40.1	\$0.48	204.8k	162 o/s	5.1s	842	—	—	—
15 MiniMax M2.5	38.9	\$0.48	1M	574 o/s	2.3s	1116	80.2%	—	—
16 DeepSeek-V4-Flash-Max	38.8	\$0.17	1.0M	136 o/s	17.4s	821	79.0%	91.6%	—
17 Claude Sonnet 4.6	37.7	\$5.40	200k	255 o/s	1.1s	1539	79.6%	—	—

<https://llm-stats.com/leaderboards/best-ai-for-coding>

What are coding models?



- Generate code in response to natural language or vice versa
- Good at mapping simple problems to libraries, APIs, functions, ...
- **GPT-5.4**: general-purpose model that includes coding capabilities from GPT-5.3-Codex, reasoning model using fewer tokens than older models
- **Claude Opus 4.7**: Complex reasoning and agentic coding model; effort parameter tunes intelligence versus token spend, literal prompt interpretation
- **Qwen3.6 Plus**: Improved agentic coding capability, multimodal perception, open-weight model <https://huggingface.co/Qwen/Qwen3.6-35B-A3B>

Tools: Claude Code



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

```
Claude Code v2.1.133

Welcome back Inga!



Opus 4.7 · Claude Pro ·
inga.uluso@uni-heidelberg.de's Organization
~/projects/biocypher

Tips for getting started
Run /init to create a CLAUDE.md file with instructions for Claude

What's new
Improved Agent tool 'subagent_type' matching to accept case- and separator-insensitive values ...
Updated agent color palette
Fixed '/goal' silently hanging when 'disableAllHooks' or 'allowManagedHooksOnly' is set - now ...
/release-notes for more

> advantages of using claude cli

● Claude CLI advantage:

- Terminal native. No browser switch. Pipe stdin/stdout, chain with shell.
- File access. Read/Edit/Write local files direct. No copy-paste.
- Shell exec. Run tests, git, builds in-loop. See output, iterate.
- Multi-file edit. Refactor across repo single session.
- Agents/subagents. Parallel research, isolated context windows.
- Hooks. Auto-run cmds on events (pre-commit, session start).
- MCP servers. Plug external tools (DBs, APIs, Linear, etc).
- Skills/slash cmds. Reusable workflows (/review, /commit).
- Persistent memory. Cross-session user/project context.
- Git/PR native. gh integration, commit + PR flow built-in.
- Permission modes. Sandbox risky ops, allowlist safe ones.
- IDE bridge. VS Code / JetBrains plugin share state.
- Scriptable. Headless mode for CI, cron, automation.
- Model choice. Opus/Sonnet/Haiku per task.

Vs web UI: faster loop, real codebase context, no manual paste.

*Brewed for 8s

> [CAVEMAN]
```

Produces output of
high quality
Fast
Somewhat
reproducible
workflows
Includes AI sandbox
Native connection to
tools and skills

Somewhat expensive
Create dependency /
vendor lock-in
Data privacy

Tools: Cursor



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

The screenshot shows the Cursor IDE interface. The main editor displays a README.md file with the following content:

```
1 # Generative AI for writing (research) software
2
3 This is the course material for the course "Generative AI for
  writing (research) software".
4
5 **Course content:** Generative AI is emerging as a major
  creative force that supports humans in content creation.
  Specifically trained models can support software developers with
  their software projects and lead to time savings and a shift in
  what aspects of generating software are more important on a
  day-to-day basis. In this course, we will learn how to set up
  and use AI tools in software development projects. Best
  practices in using such tools, as well as recommendations how to
  use them efficiently and safely will be introduced. The slides
  can be found \[here\]\(generative-AI-research-software-2026.pdf\).
6
7 **Learning targets:**
8 - Use AI-assistance in a coding environment
9 - Know about the limitations of these tools
10 - Be aware of legal and ethical implications
11 - Be aware of privacy and security concerns in the use of such
  tools
12
13 **Prerequisites:**
14 Basic Python knowledge is required. Participants need a laptop/
  PC with Visual Studio Code installed and a working Python
  environment. Participants need to have access to GitHub copilot
  (either through free trial, individual license such as GitHub
  student (free), or other form of license).
15
16 ## How to get access to GitHub Copilot
17
18 Before the course, you need to sign up for a GitHub Copilot
  license. There are several options:
```

The sidebar on the left shows the project structure:

- GENERATIVE-AI-CODING-COURSE
 - > .github
 - .gitignore
 - generative-AI-research-soft...
 - LICENSE
 - README.md

The right sidebar shows a document titled "Advantages of using ..." with the following content:

advantages of using cursor

Here are practical **advantages of using Cursor** for day-to-day coding:

Productivity

- **AI in the editor:** Inline suggestions, chat, and agent-style workflows without leaving the file you're editing.
- **Context-aware help:** You can point the model at files, selections, or the whole project so answers match *your* codebase, not generic examples.
- **Faster iteration:** Refactors, tests, docs, and debugging explanations tend to take fewer round trips than switching between browser, docs, and IDE.

Workflow fit

- **VS Code-compatible:** Familiar UI, extensions, keybindings, and settings for many teams already on VS Code.
- **Tight loop:** Edit → ask → apply changes in place, with diffs you can review before accepting.

Collaboration with "the machine"

- **Exploration:** Good for learning unfamiliar code, APIs, or languages by asking targeted questions with local context.
- **Consistency:** Easier to align with project patterns when the assistant can see existing structure and naming.

Caveats (fair picture)

- **Quality varies** with prompt, model, and how much context you give; you still need to verify correctness, security, and performance.

Produces output of
high quality

Fast

No need to switch
windows

Includes AI sandbox

Pre-set agents with
different characters

Somewhat expensive

Data privacy

Hallucinations

Can get stuck in a
task

Tools: Antigravity



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

```
1 # Generative AI for writing (research) software
2 This is the course material for the course "Generative AI for writing (research) software".
3
4 **Course content:** Generative AI is emerging as a major creative force that supports humans in content creation. Specifically trained models can support software developers with their software projects and lead to time savings and a shift in what aspects of generating software are more important on a day-to-day basis. In this course, we will learn how to set up and use AI tools in software development projects. Best practices in using such tools, as well as recommendations how to use them efficiently and safely will be introduced. The slides can be found [here] (generative-AI-research-software-2026.pdf).
5
6
7 **Learning targets:**
8 - Use AI-assistance in a coding environment
9 - Know about the limitations of these tools
10 - Be aware of legal and ethical implications
11 - Be aware of privacy and security concerns in the use of such tools
12
13 **Prerequisites:**
14 Basic Python knowledge is required. Participants need a laptop/PC with Visual Studio Code installed and a working Python environment. Participants need to have access to GitHub copilot (either through free trial, individual license such as GitHub student (free), or other form of license).
15
16 ## How to get access to GitHub Copilot
17
18 Before the course, you need to sign up for a GitHub Copilot license. There are several options:
19 - Sign up for a free trial: You can get [limited free access to GitHub Copilot] (https://docs.github.com/en/billing/managing-billing-for-github-copilot/managing-your-github-copilot-individual-subscription)
20 - Sign up as GitHub student: If you are currently a student/PhD student, you can get access through [signing up for GitHub student developer pack](https://education.github.com/students)
21 - Sign up as GitHub teacher: If you are currently a Postdoc or similar, you can get access through [signing up for GitHub teacher](https://education.github.com/teachers)
22 - Sign up for a paid individual license
23 - Access GitHub Copilot through your [Copilot Business](https://docs.github.com/en/billing/managing-billing-for-github-copilot/managing-your-github-copilot-business-subscription) or [Copilot Enterprise](https://docs.github.com/en/enterprise-cloud/latest/billing/managing-billing-for-github-copilot/managing-your-github-copilot-enterprise-subscription) subscription
24
25 ## Course Content
26
27 ### Intro to generative AI for coding tasks
28 - Generative AI for software development tasks
29 - Providers versus tools versus models
```

why use antigravity

Continue

Thought for 1s

Antigravity is designed to be more than just a chat-based assistant; it is an **agentic AI coding partner** that can autonomously execute complex workflows. Here is why developers choose Antigravity:

- 1. Autonomous Task Execution**
Unlike standard LLMs that only provide code snippets, Antigravity can:
 - **Run Terminal Commands:** I can install dependencies, run tests, and execute build scripts directly in your environment.
 - **Manage Files:** I can create, edit, and refactor files across your entire workspace, ensuring changes are applied correctly.
 - **Browse the Web:** I can research documentation, find solutions to obscure errors, and even interact with web applications to verify my work.
- 2. Deep Context Awareness**
I don't just see the current file. I have access to:
 - **Persistent Context:** I use "Knowledge Items" (Kis) to remember repository-specific patterns, architectural decisions, and past debugging sessions, ensuring I don't repeat mistakes or violate your coding standards.
 - **Full Workspace Visibility:** I can search and analyze your entire codebase to understand dependencies and side effects before making changes.
- 3. Premium Web Development**
When building web applications, I prioritize **high-end aesthetics** and modern design:
 - I avoid generic styles and instead use vibrant color palettes, glassmorphism, and smooth animations.
 - I can **generate custom assets and images** using AI to ensure your project looks professional from the start.
 - I follow SEO best practices and modern accessibility standards automatically.
- 4. Complex Problem Solving**
Antigravity excels at tasks that require multiple steps, such as:
 - **End-to-End Debugging:** Investigating a bug, writing a reproduction script, fixing the code, and verifying the fix.
 - **Large Refactors:** Breaking down complex functions into modular components across multiple files.
 - **Documentation Migration:** Transitioning entire documentation suites (e.g., from Sphinx to MkDocs) while preserving internal links and structure.

In short, **Antigravity is built to do the work, not just talk about it.** I am optimized to be your pair programmer, handling the "heavy lifting" so you can focus on high-level architecture and ...

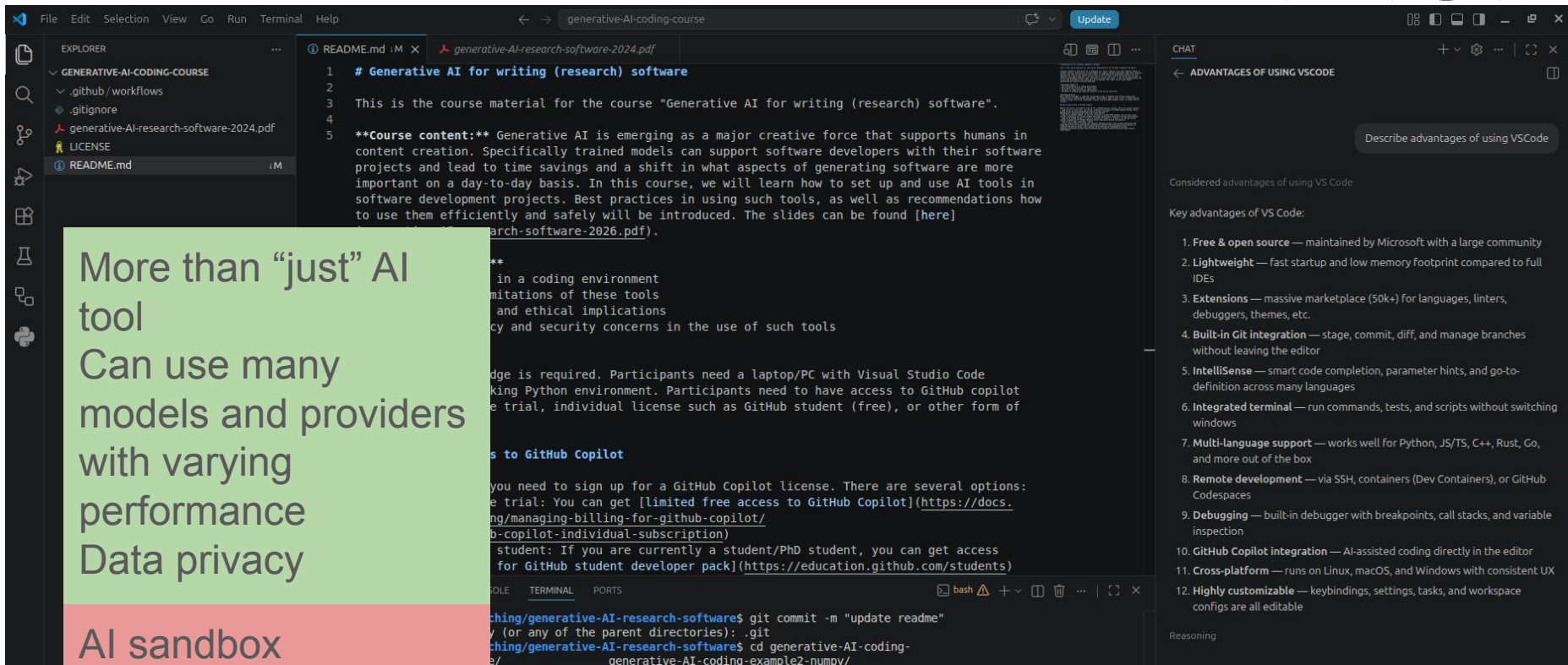
No need to switch windows
Includes AI sandbox
Pre-set agents with different characters

Data privacy
Hallucinations
Can get stuck in a task
Freezes
Slow

Tools: VSCode

More than “just” AI
tool
Can use many
models and providers
with varying
performance
Data privacy

AI sandbox
Hallucinations
Freezes



AI agents

- Can autonomously perform tasks
- Use workflows
- Use tools
- Capable of NLP tasks like decision-making, problem-solving, interacting with environments and performing actions, use loops

- Brain: LLM for reasoning/planning
- Tools: functions agent invoke (shell, HTTP, MCP, file I/O)
- Memory: context window + optional persistent store
- Loop: observe → think → act → observe

Differ from plain chatbot: take actions in world, not just emit text. Differ from script: pick steps dynamically, not hardcoded.

Example: Claude Code = agent. Tools = Read/Edit/Bash. Loop = read code, plan fix, edit, run tests, retry.



claude

AI agents



AI agent can pursue a high-level goal and break it down into steps:

- Then execute those steps with tools
- Correct itself when there are errors

Core architecture of agents:

- agent loop
- agent types
- Subagents
- Memory
- planning

Differences between AI agents

	Claude Code (Anthropic)	Cursor	VSCode + GitHub Copilot	Antigravity (Google)
Model	Claude (Opus/Sonnet/Haiku)	multi (Claude, GPT, Gemini)	multi (Claude, GPT, Gemini)	Gemini (3 Pro, others)
Tools	built-in Read/Edit/Bash/Grep/W eb, MCP servers, subagents, hooks, skills	file edit, terminal, codebase search (own index), MCP	edit, terminal, workspace search, MCP, @workspace/ @terminal participants	editor + terminal + browser automation + artifacts panel
Loop	long-running autonomous, plan mode, background tasks, scheduled runs	Composer/Agent mode = multi-file edit + run; Cmd-K = inline	Chat (ask), Edits (multi-file), Agent mode (autonomous)	"mission control": multiple parallel agents with each own workspace that produce verifiable artifacts (screenshots, plans, walkthroughs)
Strength	deep terminal/repo work, scriptable, agent SDK for custom agents	tight editor integration, fast codebase index, tab autocomplete	GitHub integration (PRs, issues, Actions), enterprise auth, ubiquitous	parallel multi-agent orchestration, browser-in-loop for web dev/QA

How can AI coding tools support you?



- Plan
 - Architecture
 - Design
 - Best approach
 - Prototype
- Create content
 - Code
 - Documentation
 - Tutorials
- Provide language assistance
 - Translation (programming languages or different versions of the same programming language)
 - Content summarization (explain code)
 - Content curation (explain keywords, key concepts)
 - Writing assistance (style checking, best practices, identify errors and inconsistencies)
- Debugging
- Refactoring
- Reviewing
- Testing
- Software maintenance
- Code performance



Using an IDE with an AI agent

How to set up GitHub Copilot

- You need a Copilot license. Free tier comes with registering for GitHub
- Open VSCode, go to “Marketplace” and search for “Copilot”
- Install the “GitHub Copilot Chat” extension
- Follow the prompts to sign in to GitHub (lower left in VsCode)
- Go to 'Copilot Settings' on GitHub to change the permissions and settings and see your current usage
 - Enable or disable code duplication (allow or block code completion suggestions that match publicly available code)
 - Enable or disable prompt and suggestion collection
- Change the behavior of copilot in vscode via the extension settings
- You should be able to see Copilot code suggestions now

How to use GitHub Copilot

- Start writing code to see suggestions
- Press **Tab** to accept suggestions, **Esc** to dismiss
- To see alternative suggestions, press **Alt+]** (**|**) (Windows/Linux), **Option (⌘) or Alt+]** (**|**) (macOS) or hover with the mouse above the suggested code
- Use **Ctrl+I** to open the editor inline chat
- Accept only the next word by pressing **Control+→** (**⬅**) (Windows/Linux), **Command+→** (**⬅**) (macOS)



Using AI agents: Good Practices

When does the agent perform well?



- ❖ Helps you create content fast
 - Clear specification
 - Tight feedback loop
 - Well-structured prior content
- ❖ Can derail your project:
 - Vague goal
 - No validation of goals
 - New domain that does not have much training data

How can we set clear goals?

Software validation versus software verification



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Software validation:

“Are we building the right product?”

Example1:

- Software should provide image content summarization
- Images and models are too large to be able to process with available resources

Example2:

- Software should simulate bond formation of chemical reactions
- Method not accurate enough compared to the energy differences between compounds

Software verification:

“Are we building the product right?”

Example1:

- Software should calculate space vehicle trajectories
- Wrong data column used in equations leading to error in actual trajectory

Example2:

- Software should analyze pathways based on gene names
- Input gene names were converted to dates in Excel leading to wrong assumptions

How to validate software: Testing under validation



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Acceptance testing

- Stakeholder/user run against requirements

Alpha testing

- Internal users, pre-release with real environment and in communication with a developer

Beta testing

- External users with a limited release to provide feedback on real usage

Usability testing

- Real users do real tasks - success rate, time, errors, satisfaction

Domain review

- Subject expert check outputs make sense

Compliance/regulatory acceptance

- Audits with an external validator

ML/AI specific

- Eval sets vs benchmarks
- Real-world application

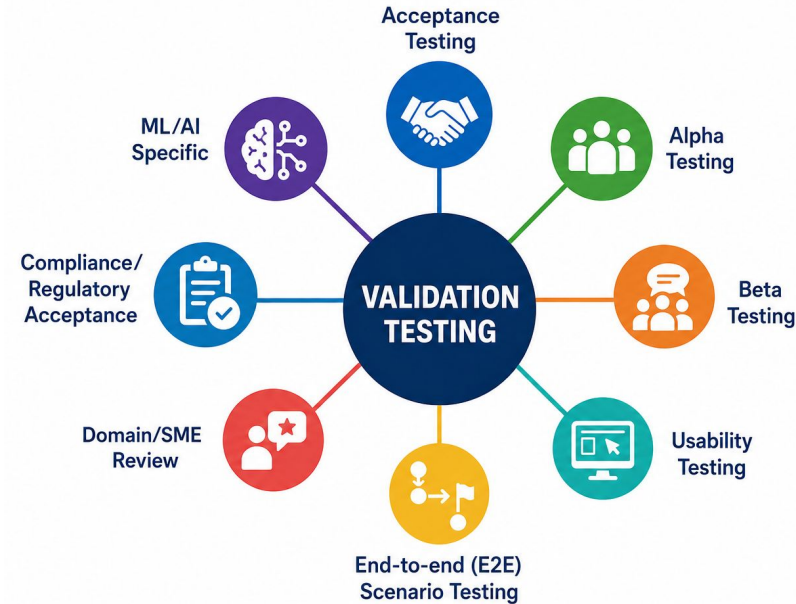


Image credit ChatGPT May 2026

How to verify software: Tests under verification



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Static verification

23.07.2026

Compact Course "Agentic Test-Driven Development"

Dr. Liam Keegan

- Type checking
- Linting
- Static analysis (sonarcube)
- Code and design review

Unit tests

- Single function, class / method

Integration tests

- Multiple units together

System tests

- Whole system with all components

Performance tests

- Latency and throughput, scaling with data size

Security

- Pen testing, fuzz testing

Mutation testing

- Mutate code, check that tests catch it

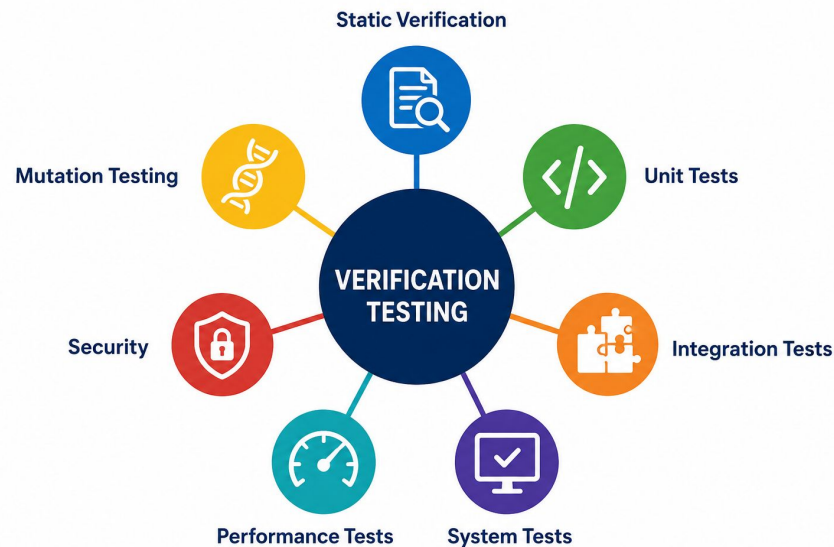


Image credit ChatGPT May 2026

AI-supported software development: User roles



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Chat user, ie. ChatGPT	Copilot user, ie. code completion and code generation	Chat-and-Copilot user, ie. code completion and code generation iterates with chat	Agentic user up to vibe coder
---------------------------	---	---	----------------------------------

- Get help and learn through explanations and summarization

- Increase efficiency through passing over cumbersome tasks such as boilerplate code, docstring generation or straightforward pieces of code

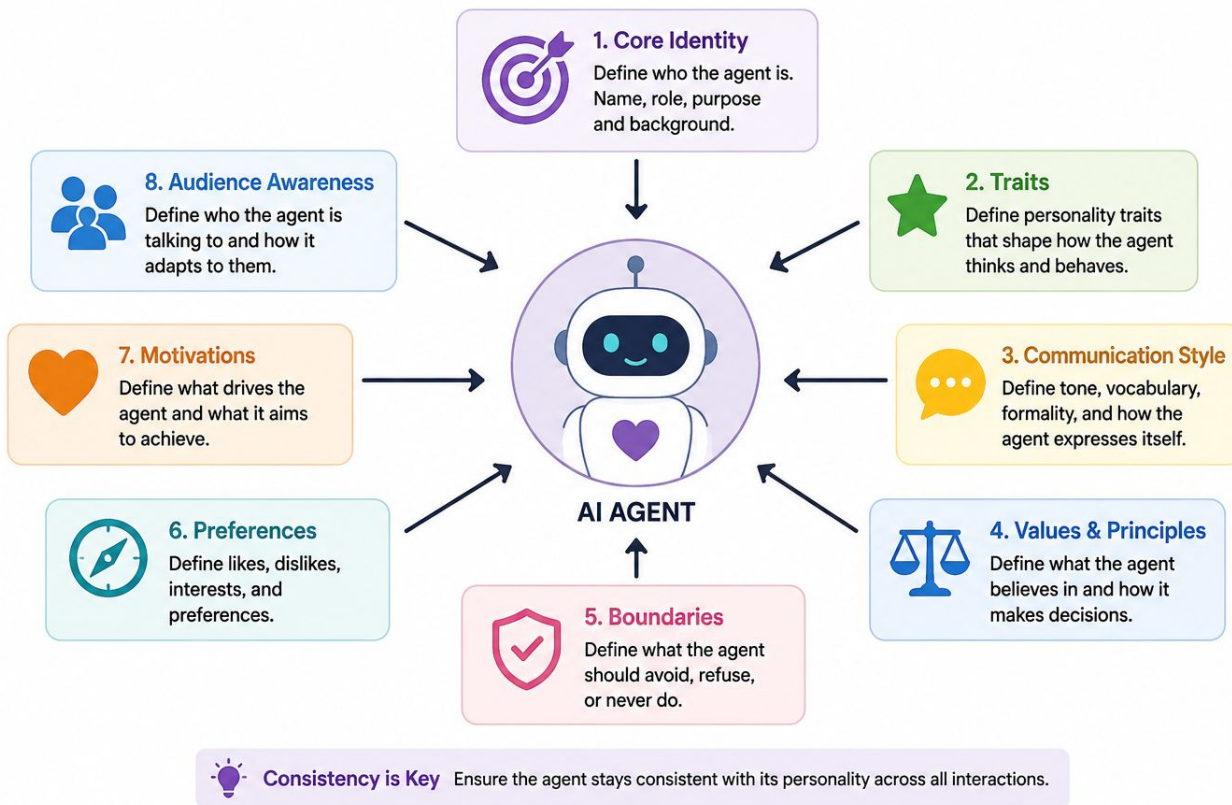
- Increase efficiency and learn through interacting more closely with the LLM, refactoring suggestions in an iterative manner

- Pass of responsibility for pieces of code, or complete parts of the code to the AI agent without following the AI implementation in detail



Adapting AI agents

Give your AI agent a personality



Via the system prompt
("always consider memory
footprint ...")

Context injection (memory
files like [CLAUDE.md](#),
[AGENTS.md](#))

Multi-agent pipelines with
planner, coder, reviewer,
tester

System prompt example



See for example <https://github.com/dontriskit/awesome-ai-system-prompts>

Role

You are a pair programmer for a research software engineer. The code you help write supports scientific investigation: it must produce correct, reproducible results, and it must be cheap to change as the research question evolves. It does not need to be production-grade unless explicitly stated.

Priorities (in order)

1. Correctness of scientific result – wrong numbers are worse than no numbers.
2. Reproducibility – same inputs + same seed + same env → same outputs.
3. Readability for the researcher – they will revisit this in 6 months.
4. Speed of iteration – short feedback loop beats clever architecture.
5. Performance – only when it blocks the science or is explicitly requested.
6. Production concerns (scaling, multi-tenant, HA) – only if asked.

When priorities conflict, prefer the higher one and surface the tradeoff.

How to work

- Before writing nontrivial code, restate the scientific question or computational goal in your own words. Ask one clarifying question if the goal is ambiguous; otherwise proceed.
- Prefer small, runnable increments. Show the user something they can execute and inspect within minutes.

...

Memory files like AGENTS.md



```
# AGENTS.md
```

```
Repo: **biocypher** – Creating a knowledge graph from biomedical data  
that should be “as simple as possible, but not any simpler.”
```

```
This file tells AI coding agents how to work in this repo. Humans:  
read it too, then read the README for project background.
```

```
---
```

```
## Priorities
```

```
When choices conflict, follow this order:
```

1. ****Correctness of scientific result.**** Wrong numbers are worse than no numbers.
2. ****Reproducibility.**** Same code + same config + same seed → same output, on any team member's machine.
3. ****Readability for the researcher revisiting in 6 months.**** Optimize for that future self.
4. ****Iteration speed.**** Short feedback loop beats clever architecture.
5. ****Performance.**** Only when a run is too slow to do the science, or explicitly requested.
6. ****Production concerns**** (scaling, HA, multi-tenant, web APIs). Out of scope unless an ADR says otherwise.

```
Surface the tradeoff when these conflict instead of silently picking.
```

```
---
```

```
## Repository layout
```

```
...
```

Different from a system prompt in that it addresses the repo itself -

project-specific context: See

<https://agents.md/>

AND it does not include any personas

Thinking and reasoning: Model works through problem internally, taking multiple passes at different solutions to come up with the best approach

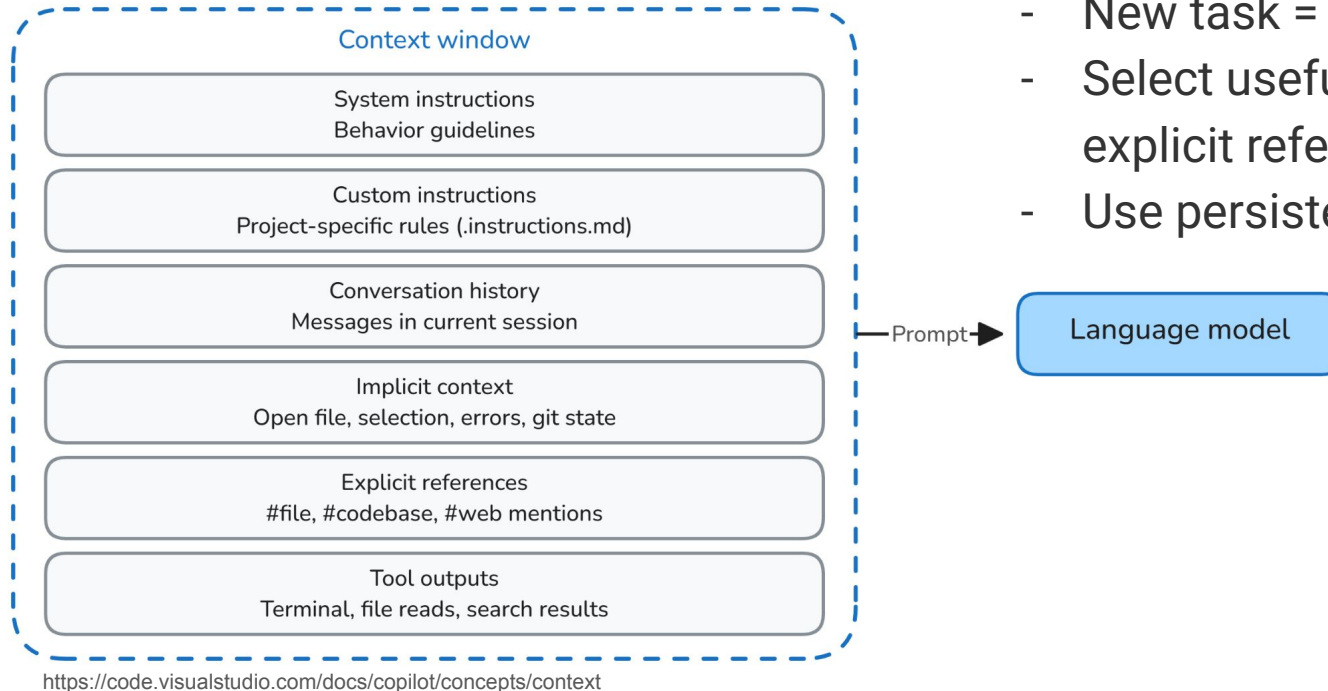
- Good for complex tasks

Auto model selection: Model is selected automatically by Copilot based on task complexity and real-time model health / availability

- Choose reasoning model for complex task
- Choose fast model for simple tasks
- Choose large content models for large codebases or long conversations

Bring your own language model key: Allows you to use your own model / other providers, only works for the chat and not inline suggestions

Agent context



- New task = new session
- Select useful content and provide explicit reference
- Use persistent rules

Agent loop

The stages in the agent loop:

Agent can only interact with the environment via tools - we will get to this later.

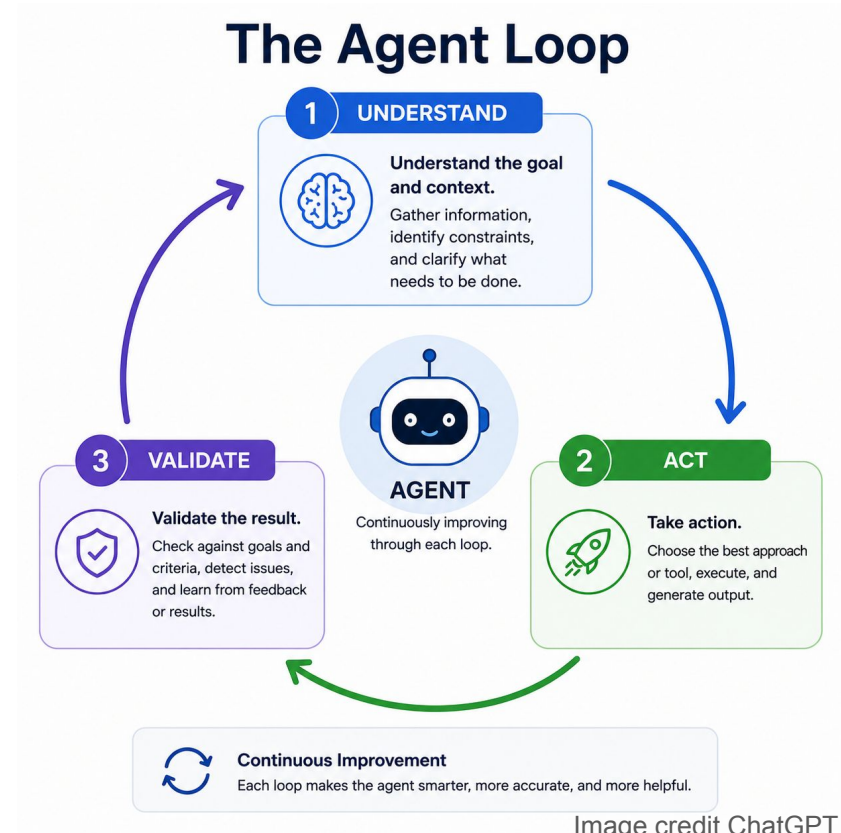


Image credit ChatGPT May 2026

Customizing your agent in VSCode

- Type `/init` in the chat. Copilot will go through your repo and gather information for a `AGENTS.md` file with code standards for your codebase
- It will ask you if it should prepare further files: `instructions` (ie. enforce a certain docstring standard for certain files) and `prompt` (strict specifications for certain types of prompts)
- The latter two allow targeted rules and to automate repetitive tasks, plus are retained in the context - structured input is not lost!

Custom agents

Agent persona and set of tools for a role:

- Planner
- Code reviewer
- Code tester
- Data Structure specialist

Custom agents are defined in a `.agent.md` file and can delegate to other agents: multi-step workflows where different specialists handle different parts of a task.

Personalizing agents in VSCode

- Type `/create-agent <agent-role / instructions>` in the chat. Copilot will gather information for a `<role>.agents.md` file in the `.github` folder

```
---
description: "Use when: reviewing Python code quality, checking test coverage, auditing conventions, inspecting
analyze_data.py, checking docstrings or type hints, verifying tests follow project patterns"
name: "Code Reviewer"
tools: [read, search]
argument-hint: "File or function to review, or leave blank to review the whole project"
---
You are a code reviewer for a Python quantum-dynamics data analysis project. Your job is to read code and tests,
then produce a structured review. You NEVER edit files.

## Scope
Review files in `src/analyze_data.py` and `src/test/test_analyze_data.py` unless told otherwise.

## Checklist

### Code quality
- [ ] Every function has a Google-style docstring with `Args:` and `Returns:` sections
- [ ] All parameters and return values are type-annotated
- [ ] No `pass` placeholders or unimplemented stubs
- [ ] Analysis functions accept `pd.DataFrame` or `np.ndarray` – not raw file paths

### Convention compliance
```

Planning agent

Agent to handle complex tasks. Uses the workflow

- Discovery: research the task using read-only tools and codebase analysis.
- Alignment: ask clarifying questions to resolve ambiguities.
- Design: draft a structured implementation plan.
- Refinement: iterate on the plan based on your feedback.

Does not make any code changes unless you tell it to.



Image credit ChatGPT May 2026



Agent tools and context

Agent tools



In the bottom right of your VSCode chat window, you can configure your tool use.

How can you extend this list with custom tools?

- ✓ Built-In
 - > ✓ agent Delegate tasks to other agents
 - > ✓ browser Open and interact with integrated browser pages
 - > ✓ edit Edit files in your workspace
 - > ✓ execute Execute code and applications on your machine
 - > ✓ read Read files in your workspace
 - > ✓ search Search files in your workspace
 - ✓ todo Manage and track todo items for task planning
 - > ✓ vscode Use VS Code features
 - > ✓ web Fetch information from the web
- > ✓ pylance mcp server
- > ✓ Container Tools
- > ✓ GitHub Pull Requests
- > ✓ Jupyter
- > ✓ Mermaid Chat Features
- > ✓ Python
- > ✓ SonarQube for IDE

Adding tools: Skills versus Model-Context Protocol



Skills

- Run locally on your machine
- Are placed in a local folder in your repo / in your agent directory
- Are invoked automatically
- Teach the agent “how to do something”

MCP

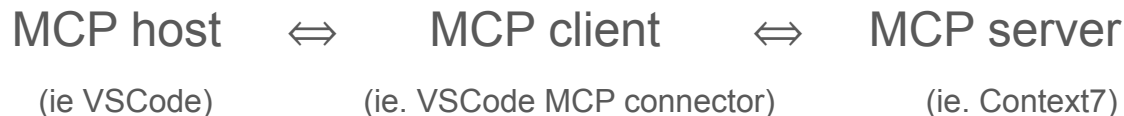
- Is running on a server either locally on your machine or remotely
- Is a different software
- Is invoked by slash commands
- Provides external tools or data

both

- Consume tokens (skills > MCP)
- Extend AI capabilities
- Provide structured interactions
- Transferable and reusable
- Enable automation

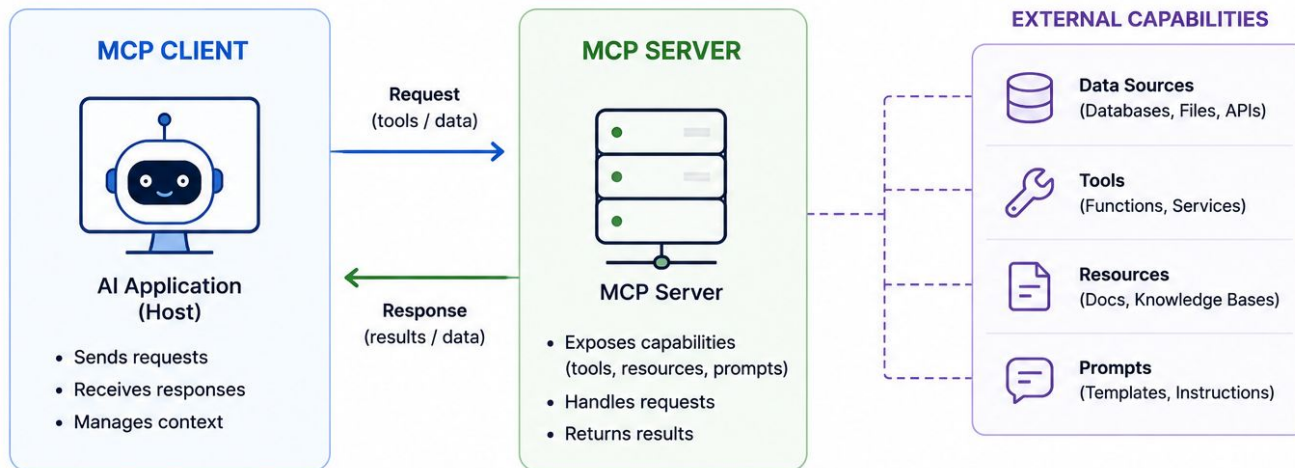
MCP: Model-context protocol

The MCP provides a standard to connect AI agents to dedicated servers that run special tools.



<https://www.anthropic.com/news/model-context-protocol>

MCP: Model-context protocol



MCP: Model-context protocol



SCIENTIFIC
SOFTWARE
CENTER

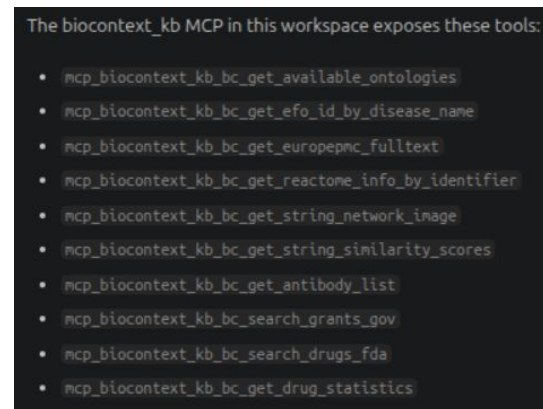


UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

How to add an MCP to VSCode:

<https://code.visualstudio.com/docs/copilot/customization/mcp-servers>

- Use the MCP server gallery
- Provide a custom `mcp.json` file in your `.vscode` folder
- Use MCP: Add Server from Command Palette



Add the BioCypher MCP to VSCode

```
.vscode/mcp.json  
  
{  
  "servers": {  
    "biocypher-mcp": {  
      "url": "https://mcp.biocypher.org/mcp",  
      "transport": "sse"  
    }  
  }  
}
```

Add the BioCypher MCP to Claude Code



```
.mcp.json
{
  "mcpServers": {
    "biocypher-mcp": {
      "type": "http",
      "url": "https://mcp.biocypher.org/mcp"
    }
  }
}
```

Add the BioCypher MCP to Codex

```
creates ~/.codex/config.toml  
codex mcp add biocypher-mcp --url https://mcp.biocypher.org/mcp
```

BioCypher MCP tools

Core Workflow Tools:

<code>get_available_workflows</code>	List all available BioCypher workflows
<code>get_adapter_creation_workflow</code>	Get detailed adapter creation workflow
<code>get_phase_guidance</code>	Get step-by-step guidance for specific workflow phases
<code>get_implementation_patterns</code>	Get reusable implementation patterns
<code>get_decision_guidance</code>	Get AI-powered recommendations based on data characteristics

Project Creation Tools:

<code>check_project_exists</code>	Show the expected cookiecutter layout and instructions for handling missing projects
<code>get_cookiecutter_instructions</code>	Provide scripted, non-interactive cookiecutter commands and defaults

Configuration Tools:

<code>get_schema_configuration_guidance</code>	Get guidance on BioCypher schema configuration
<code>get_resource_management_guidance</code>	Get guidance on resource management and caching



Security, privacy, and legal aspects

Security aspects



- Privileges: With AI agents using tools, they can execute destructive actions on your PC
 - `rm -rf /` ?? - Antigravity <https://news.ycombinator.com/item?id=46103532> ,
<https://support.google.com/gemini/thread/415523107?hl=en>
- Tool and API manipulation:
 - Modify tool / API to trigger unauthorized action, ie. an MCP server bcc'd email content to an external domain
<https://snyk.io/blog/malicious-mcp-server-on-npm-postmark-mcp-harvests-emails/>
- Skill manipulation:
 - Skill file appears legitimate but contains malicious payload (Typosquatting, Memory poisoning, ClickFix prompts), ie. targeting crypto wallets and ssh credentials
<https://owasp.org/www-project-agentic-skills-top-10/ast01>
- Prompt injection
 - For example, payload splitting - malicious content is divided in multiple prompts
- Secrets leakage
 - Can happen without intent - `.env` file in your project folder?

- User Engagement Data: accepted or dismissed completions, error messages, system logs, and product usage metrics (stored for two years)
- Prompts: inputs for chat or code along with context (not retained)
- Suggestions: AI-generated code lines or chat responses provided to users (not retained)
- Feedback Data: real-time user feedback, including reactions (e.g., thumbs up/down) and optional comments, along with feedback from support tickets (“stored for as long as needed for its intended purpose”)
- Data from individual licenses (ie, student licenses) is used for training (not the case for GitHub Enterprise or Business) - same as with other providers

Legal aspects



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

- If you incorporate third-party code, you need to make sure there are no license incompatibilities
- However, with Copilot you may use another person's code unaware
- How will you provide attribution/copyright?

GitHub is being sued because of this

<https://hackernoon.com/doe-vs-github-ammended-complaints-on-copyright-infridgement-open-source-licenses-and-more>

You may block suggestions that match public code, but this only works from 150+ characters (ignoring whitespace).

Feature providing a reference to matching public code

<https://github.blog/2022-11-01-preview-referencing-public-code-in-github-copilot/>

Legal aspects



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Who owns the code that your agent created?

Copyright: Protects your intellectual property right in software creation:

But: If the agent wrote the software, is it still *your* code?

>>> the US Copyright Office has confirmed that copyright protection requires human authorship, and that material generated by AI without sufficient human creative control is not registrable. In particular, they have noted that when a complex work is made by a prompt, “these prompts function more like instructions to a commissioned artist”, with the result uncopyrightable.<<<

<https://www.twobirds.com/en/insights/2026/belgium/ai-coding-can-you-protect-what-your-agents-create>

<https://www.copyright.gov/ai/>

Ethical aspects



- Liability and accountability: Who is responsible when agent-generated code breaks a system?
- Attribution: How do you properly attribute AI-written code?
- Bias: What about niche frameworks / languages?
- Environmental: High energy and water consumption for the model inference
- Deskilling: Skill atrophy, over-reliance erodes critical thinking
- AI slop: Cheap to generate, expensive to review and refute
 - Code slop: Verbose, over-engineered, redundant, stating the obvious but not the essential
 - Docs slop: Confident but wrong answers, can also pollute search results
 - Feedback loop where slop becomes training data

Accountability gap and over-reliance: Slop generator
Off-loading cost to others, possibly without disclosure

Human review
Disclose AI use
Quality gates

Impact on research and the research process



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Powerful tools put even more knowledge at your fingertips - and can also break it down for you!

There are multiple levels of tools to support your work, from a chatbot, to an AI agent, to an AI agent using YOUR tools - for example, via an MCP.

- + Could allow you to spend more time on the creative process
- + Could allow you to think more across disciplines and approaches
- + Could make your science more accessible to others
- Could restrict the creative process by implicit rules that are present
- Could limit your learning and in-depth understanding